CPS721: Assignment 1

Due: October 1, 2026, 11:59pm
Total Marks: 120 (worth 4% of course mark)
In groups of no more than 3

Late Policy: Late submissions are penalized by 10% per day.

Clarifications and Questions: Please use the discussion forum on the D2L site to ask questions. These will be monitored regularly. A Frequently Asked Questions (FAQ) page will also be created. Only ask questions by email if it is specific to your submission. Any questions asked with less than 24 hours left to the deadline will not be answered.

PROLOG Instructions. Your code must run on ECLiPSe 7.1. If you cannot install this on your own machine, test it on the lab machines or the department moon servers.

You are NOT allowed to use ";" (disjunction), "!" (cut), "->" (if-then), `setof`, `bagof`, `findall`, or any other language features and library predicates that are not covered in class. Similarly, you should only use "=" and "is" for equality, and "not" for negation. Do NOT use "=:=", "\=", or other such commands. You will be penalized if you use these commands/features. You are only allowed to use ";" to get additional responses when interacting with PROLOG from the command line (this is equivalent to using the "More" button in the ECLiPSe GUI).

Academic Integrity and Collaboration Policy: Any submission you make must be the work of your group and your group alone. You are also not allowed to post course materials (including assignments, slides, etc.) anywhere on the web, and you are also prohibited from using generative AI systems or online "help" websites for completing assignments.

Submission Instructions: You should submit ONE zip file called `assignment1.zip` on D2L. It should contain the following files:

```
assignment1_submission_info.txt
q1a_trip_kb.pl        q1b_queries.pl         q1_log.txt
q2a_estore_kb.pl      q2b_estore_calc.pl     q2_log.txt
q3_trip_planning.pl   q3_log.txt
```

All these files have been given to you and you should fill them out using the format described. Ensure your names appear in all files — especially `assignment1_submission_info.txt` — and your answers appear in the file sections as requested. Do NOT edit, delete, or add any lines that begin with "`%%%%% SECTION:`". These are used by the auto-grader to locate the relevant code. You will lose marks if you edit such lines.

Your submission should not include any other files than those above. You will be penalized for using compression formats other than `.zip`. Submissions by email will not be accepted.

You may submit your solution multiple times. Doing so will simply overwrite your previous submission. We will therefore always mark the last submission your group makes.

1 A PROLOG Knowledge Base [40 marks]

For this problem, you will be making a simple knowledge base (KB) containing flight, hotel, and attraction information. The intention of the knowledge base is to provide information for an autonomous agent to do trip planning, on behalf of a human who is trying to plan a vacation. In part (a), you will be creating a knowledge base. In part (b), will formulate queries to answer questions about that knowledge base. Finally, in part (c), you will test your queries and knowledge base and report the results.

a. [10 marks] Create a knowledge base by adding atomic propositions to the file `q1a_trip_kb.pl`. The KB should ONLY contain atomic statements that use the following predicates:

- `flight(City1, City2, Date, Price)` - there is a flight from `City1` to `City2` on `Date`, whose cost is `Price`.
- `hasRoom(Name, City, Date, Price)` - the hotel with the given name in the given city has availability on the night of `Date`. The cost for that night is `Price`.
- `ticket(Name, City, Date, Price)` - there are tickets available for an attraction called `Name` (e.g. the CNE) that takes place in `City` on `Date`. The cost is `Price`.

Your knowledge base should be added to the file `q1a_trip_kb.pl` in the `q1_kb` section. See the provided file for formatting instructions. That file should include at least 10 statements for each of `flight`, `hasRoom`, and `ticket`, but you may add more to help you better test your queries in part (b). In particular, it is probably a good idea if your KB is set up so that your queries in part (b) each succeed at least once.

Notice that we have made a few simplifying assumptions:

- For `flight`, there will only be a single flight from `City1` to `City2` on any day. There should be no flights from a city to itself.
- There may be hotels with the same names in different cities. However, there should only be one `hasRoom` statement for any combination of hotel name, city, and date in the KB (*ie.* there are not two different statements for the Sheraton in Halifax on September 5th). This corresponds to assuming there is only one type of room for each hotel. The price of a room at a hotel may differ from day to day.
- Similarly, there is only one ticket price for any attraction on any one day. However, the price may differ from day to day.
- We will assume that we are only trip planning in October, so all dates will be given as an integer indicating the day in October. For example, October 11 will be specified as 11.
- All prices will be given as positive integers.

Finally, remember that all city, hotel, and attraction names should be in lower case. In your KB, any spaces in names or locations should therefore be replaced by underscores. For example, you may have a flight from `toronto` to `quebec_city`. Do NOT use strings like "Quebec City".

b. [25 marks] Create queries for each of the 12 statements below and add them to the file `q1b_queries.pl`.

1. (0 marks) What is the price of the flight from Toronto to Quebec City on October 9? *Note, the answer for this question has been given to you as a way to show you the expected formatting.*
2. (1 mark) Does the Sheraton in Halifax have availability on October 4?
3. (1 mark) Find an attraction in Calgary that has tickets available for it on October 18.

4. (2 marks) On what date can you get opera tickets in Vancouver for less than $75?
5. (2 marks) What is the total cost of staying at the Hilton in Saskatoon on October 17, 18, and 19 (ie. checking in on October 17 and checking out on October 20)?
6. (2 marks) Find a city other than Victoria and Banff, that you can fly to from Vancouver on October 27, where the cost is less than $300.
7. (2 marks) I live in Montreal and want to take a 3 night vacation in another city. Find a city I can fly to and back from for no more than $500 (not including hotels). The dates should be between October 20 and 31st (inclusive). Note that a 3 night vacation means the return flight is 3 days after the first flight (*e.g.* flights on the 21st and 24th).
8. (2 marks) Find a city that has more than one hotel with availability on October 27.
9. (3 marks) Find a day in October that the Holiday Inn in Toronto does not have an available room.
10. (3 marks) Is there an attraction in Vancouver that has tickets available for any 3 consecutive days in October?
11. (3 marks) What is the most expensive attraction that has tickets available for it in Montreal on October 19?
12. (4 marks) I live in Toronto, and am looking to take a 2 night vacation in some other city between October 10 and 20 (inclusive). Find me the cheapest such vacation. The package must include flights to and from Toronto, a hotel booking, and one attraction. Both nights must be at the same hotel, and the attraction has to be on the second day of my trip. For example, if the departing flight is on day `12`, then the same hotel has to have availability on days `12` and `13`, the attraction tickets are for day `13`, and the returning flight is on day `14`.

Please note the following before completing this question:

- Ensure that you follow the instructions for formatting as specified in `q1b_queries.pl`.
- You can then test your queries by compiling this file. It is already set to import the KB file in part (a), so you don't need to first compile that file as well.
- Your queries should capture the **logic** of the given statement, not just an answer specific to your knowledge base. In particular, we will test your queries on different knowledge bases to make sure they do not only work in your KB. You thus may find it helpful to test on different KBs.
- You should NOT introduce any helper predicates for this question.
- It is ok if your queries return the same result multiple times when you ask for more solutions. However, they should not return incorrect solutions.
- You can only use the predicates listed above with variables or constants as arguments, along with conjunction (","), equality ("="), and "not" (*ie.* negation) in your queries. You may also use $<, >, =<$, or $>=$, which PROLOG uses for less than, more than, etc. Ensure you also follow the Prolog instructions listed on page 1.
- Recall that when using the predicate `X` $<$ `Y`, both `X` and `Y` must be instantiated before the comparison. Similarly, be careful when using "not" on an expression that includes a variable.
- Notice that the facts for `day` have already been provided to you in `q1b_queries.pl`. These facts may be useful for some of the queries. Do NOT edit this section of the file.

c. [**5 marks**] Test your queries from part (b) on your KB, and log the output. You should put the log of this interaction in the `q1c_log.txt` file. This can simply be a copy paste of the 12 queries and their answer. Please ensure the difference between the queries and the answers are clear, and use extra space to separate one query from the next.

For at least 2 of the queries, you should also call "More" (or ';' in the terminal) to find all answers to the given query. Include the results of all queries even if you don't successfully complete all of them.

2 Arithmetic in PROLOG [30 marks]

This question will give you practice doing numerical calculations in PROLOG using the example of calculating the cost of buying electronics. In part (a), you will create a KB for the store and an order. In part (b), you will create rules for calculating the cost of the given order. Finally, in part (c), you will test your queries and knowledge base and report the results.

a. [**10 marks**] You will be calculating the bill at an e-store on three products: `laptop`, `monitor`, and `keyboard`. For this step, you should create a knowledge base that contains atomic statements that use the following predicates:

- `cost(Product, Cost)` - defines the `Cost` of each unit of `Product`.
- `numPurchased(Product, Count)` - indicates the number of units (`Count`) of `Product` that have been purchased.
- `shippingCost(Product, Cost)` - indicates the `Cost` to ship each unit of `Product` when using standard shipping. For example, if the cost of shipping a laptop is \$12, then `Cost` should be 36 if three laptops are purchased and `Product = laptop`.
- `expressShippingRate(Rate)` - sets the `Rate` for using express shipping. For example, if it is set to 0.25, and the cost for regular shipping of the given order is \$10, then the cost of express shipping is \$12.50. This value should be no smaller than 0, and no larger than 1.
- `taxRate(Rate)` - sets the tax rate. If it is set to 0.13, that means the tax rate is 13%. This value should be no smaller than 0, and no larger than 1.
- `freeRegularShippingMin(Amount)` - sets the minimum `Amount` of pre-tax purchases a user must make in order to get free 7 day shipping.
- `freeExpressShippingMin(Amount)` - the minimum `Amount` of pre-tax purchases a user must make in order to get free express shipping.

There should be exactly one `cost`, `numPurchased`, and `shippingCost` statement for each of the three products. You may set the values for the costs and counts to any non-negative integer values of your choosing. There should also be exactly one statement for each of the other 4 predicates.

All atomic propositions should be added to the `q2_kb` section of `q2a_estore_kb.pl`. See that file for format instructions.

b. [**15 marks**] Create rules that accomplish the following:

- Add a rule `subtotal(Sub)` which calculates the sum of the costs (`Sub`) of all three products being purchased, not including tax and shipping. This will depend on the cost of each product and the number of units of each that is purchased. For example, if the laptop costs \$1000, the monitor costs \$500, the keyboard costs \$75, and three of each item are purchased, then `subtotal(S)` should succeed with `S = 4725` since $3 \cdot 1000 + 3 \cdot 500 + 3 \cdot 75 = 4725$.
- Add rule `costWithShipping(ShippingType, Cost)` which calculates the pre-tax cost of all items in the order, including the shipping cost. For this purpose, the rule takes in `ShippingType`, the value of which is either `regular` or `express`. This determines the type of shipping to calculate for. The cost for shipping should thus take into account both the shipping type, and whether or not the customer has earned free shipping. In the case of express shipping, the shipping cost should be increased as determined by `expressShippingRate`. Note that this express shipping rate is only applied to the shipping cost, not the cost of the products themselves. `Cost` should then be the cost of purchasing all items (*ie.* the subtotal calculated using the previous rule) plus the cost of shipping them.

- Add a rule `totalCost(ShippingType, Cost)` that calculates the total cost for all the products including shipping and tax. The type of shipping is determined by `ShippingType`. Note that tax applies to both the cost of the products and the shipping cost as well.

All rules should be added to the file `q2a_estore_calc.pl` in the `q2_rules` section. You will lose marks if you do not put the code in the correct place. Importantly, we will be testing these rules using a variety of knowledge bases, so make sure they capture the logic of the rule, and do not just work correctly when using the values particular to your knowledge base.

Note that we will test your rules using our own KBs, each of which will satisfy the conditions specified in Q2a. Your rules should thus capture the logic described above, not just work for your KB. That is, your rules can assume that list of products always consist solely of `laptop`, `monitor`, and `keyboard`, but the counts, costs, and rates may differ.

c. [5 marks] Create at least 5 queries that test the `subtotal`, `costWithShipping`, and `totalCost` rules. Any queries of your choosing on these rules is allowed, as long as each rule is tested at least once. You should also add a comment above each query stating what the query checks in plain English.

The queries should be written in the file `q2c_log.txt`. At the end of this file, submit your interaction with the knowledge base as well.

3 Recursive Rules [50 marks]

Some travellers like to have multiple stops on their journey. For this question, you will write several programs that will allow them to determine if different kinds of multi-stop journeys are possible, and what the cost of doing so will be.

In part (a), you will write a recursive program to calculate the cost of staying at a hotel for a given amount of time. In part (b), you will write a recursive program to find possible round trips and the cost of those round trips.

Note, you are NOT allowed to use Prolog lists for any of the problems below. In addition, make sure you are familiar with the Prolog restrictions outlined on page 1 of this document.

a. [**10 marks**] Write a program that will take in a hotel, along with checkin date and checkout date, and calculate the cost of staying in the given hotel for those days. That is, you should implement the following:

- `stayCost(Hotel, City, CheckinDate, CheckoutDate, Cost)` - this query will hold if `Cost` is the cost of staying in `Hotel` in `City` when checking in on `CheckinDate` and checking out on `CheckoutDate`.

For example, if the cost of staying in the Hilton in Toronto is $200 on the nights of the 2nd, 3rd, and 4th, and then $250 on the nights of the 5th, 6th, and 7th, then the following holds:

```
stayCost(hilton, toronto, 3, 6, 650).
```

since the hotel costs $200 on each of the 2nd and 3rd, $250 on the 5th, and checkout is on the 6th.

In addition, please note the following:

- As in part (a), we will assume all trip planning is occurring in October, so `CheckinDate` and `CheckoutDate` will be specified as integers from 1 to 31.
- The check in date must be before the check out date. Otherwise, the query should fail.
- The hotel in question must have room on all days from the `CheckinDate` (inclusive) up until the day before `CheckoutDate`. If it doesn't, the query should fail.
- You can always assume that in any query, `CheckinDate` and `CheckoutDate` will be given as non-negative integers (*ie.* `stayCost` will never be called with `CheckinDate` and `CheckoutDate` as variables)
- For full marks, your program should also handle queries like `stayCost(H, toronto, 4, 7, C)`, `stayCost(hilton, L, 5, 10, C)`, and `stayCost(H, C, 11, 19, C)`. That is, any combination of `Hotel` and `City` may be set as a variable.

Write your program in the file `q3_trip_planning.pl` in the section `q3a_stay_cost`. See that file for formatting and testing instructions.

b. [**30 marks**] Now you will write a program to find possible multi-city round trips and their cost. That is, you will write the following program:

- `roundTrip(City, Start, End, NumFlights, Cost)` - this query will hold if there is a sequence of exactly `NumFlights` flights starting in `City` and ending back in `City`. The first flight must be on day `Start` and the last flight must be on day `End`. For every day from `StartDate` to the day before `EndDate`, there must be an available hotel. The total cost of flights and hotels will be `Cost`.

For example, suppose our knowledge base contains a $200 flight from Toronto to Montreal on the 15th, a $150 flight from Montreal to Quebec City on the 17th, and a $250 flight from Quebec City to Toronto on the 20th. Moreover, assume there is a hotel in Montreal that costs $195 on each of the 15th and 16th, and a hotel in Quebec City that costs $125 on each of the 17th, 18th, and 19th. Then the following holds:

```
roundTrip(toronto, 15, 20, 3, 1365)!
```

since $200 + 2 \cdot 195 + 150 + 3 \cdot 125 + 250 = 1365$.

In addition, please note the following:

- Again, we will assume all trip planning is occurring in October, so `Start` and `End` will be specified as integers from 1 to 31.
- The sequence of flights must be "legal". This means, they must be in chronological order (*ie.* the first flight must occur before the second flight). In addition, if the first flight is from some `C1` to some `C2`, then the second flight must be from `C2` to some `C3` (and so on).
- For simplicity, only one flight can be taken per day.
- There may be an arbitrary number of days between flights (*ie.* a multi-day layover).
- For every day from one flight to the next, there must be a hotel available, and that hotel cannot change until the trip moves to another city. That is, we want to stay in the same hotel for our entire time in each city.
- For this assignment, your program should allow for a "looping" trip, and so it is ok if you return back to the same city multiple times throughout the journey.
- Notice that you don't need to find a hotel on the night of `End` when the trip returns to `City`.
- We will assume all trips involve going somewhere other than `City`, and so any query with `NumFlights < 2` should fail.
- You can assume in all tests that `NumFlights`, `Start`, and `End` will be input as integers (*ie.* `roundTrip` will never be called with `NumFlights` being set as a variable).
- For full marks, your program should handle queries like `roundTrip(City, 3, 1, 5, C)`, meaning find a round trip from any city. When "More" is called, this should find all such flights. However, we suggest you only try to extend your solution to handle this case once you have all other conditions handled.

Your rules should be put in `q3_trip_planning.pl` in the `q3b_round_trip` section. See that file for formatting instructions.

d. [10 marks] Create 5 queries for each of parts (a), (b), and (c), that test your rules on the KB from question 1. Enter these queries, along with an English language explanation of each, in the file `q3c_log.txt`. You may include more queries if you choose, just ensure the same format is followed.

In addition, put a log of your tests at the end of this file. Include the queries and interaction even if you don't successfully complete all rules.